BCS2420
Lab 3: Operation LAST SIGNAL

Alexia-Madalina Cirstea

February 2026

Purpose of These Labs

These labs accompany Lecture 5, which introduces malware, stealth techniques, and rootkits. Rather than detecting malware using signatures or scanners, these challenges train you to **think like an analyst**: questioning what you see, validating assumptions, and understanding how malicious software hides its presence.

In Lecture 5, you learned that malware often:

- Hides itself from user tools
- Alters system call outputs
- Prunes results before returning them
- Evades detection by modifying what the user sees

These behaviors are especially characteristic of **rootkits** and stealth malware .
The challenges simulate these ideas for you in a controlled, safe environment.

1 Challenge 1: Last Signal

Narrative Context

You enter an abandoned kernel terminal inside a collapsed bunker. The system appears mostly empty. However, you are informed that a final emergency signal exists.

Your task is to locate and read that signal.

Key Learning Goals

This challenge introduces the following concepts:

- File system navigation
- Hidden files
- Environment variables
- Trusting vs. verifying system output
- Stealth behavior

Concepts from Lecture

In Lecture 5, malware is described as intentionally designed to avoid detection and resist removal . Rootkits in particular hide their presence by intercepting system calls and modifying results before they reach the user.

This challenge simulates that idea: what you see is not always what exists.

1.1 Commands Introduced

`ls`

`ls` lists the contents of a directory.

It answers questions such as:

- What files exist here?
- What folders exist here?

In normal conditions, `ls` shows what is present. However, malware can modify or filter what it shows.

`pwd`

`pwd` stands for *print working directory*. It tells you where you currently are in the file system.

Understanding your location is essential for navigation and forensic reasoning.

`ls -la`

The flags:

- `-l`: long format (permissions, ownership, size)
- `-a`: include hidden files (files starting with a dot)

Hidden files are often used for configuration, logs, or stealth storage.

Hidden Files

Files starting with a dot (`.`) are hidden by default. This is a basic concealment method used both legitimately and maliciously.

1.2 Environment Variables

Environment variables store configuration values that affect how programs behave.

They can control:

- Paths
- Libraries
- Program behavior
- Output formatting

Malware can use environment variables to alter how tools behave without modifying the tools themselves.

`printenv`

Displays all environment variables.
This is a diagnostic command that helps you understand what influences your shell.

`env -u`

This allows you to run a single command **without** a specific environment variable.
This mirrors real-world malware analysis: you isolate components to see what changes.

2 Challenge 2: Last Broadcast

Narrative Context

The bunker kernel has suffered further corruption.
Not only are files hidden, but now the system **actively censors information**.
Some tools lie.
Some outputs are falsified.
Your mission is to recover the last broadcast.

Key Learning Goals

This challenge expands upon Challenge 1:

- Stealth malware behavior
- Output manipulation
- Cross-checking system views
- Trust boundaries
- Forensic skepticism

Correlation to Lecture 5

In Lecture 5, you learned that rootkits and stealth malware often:

- Hook system calls
- Modify returned results
- Prune outputs
- Hide files, processes, and logs

This is called **postprocessing** or **result pruning** .
This challenge simulates this idea.

2.1 Commands Revisited

`cat`

`cat` outputs the contents of a file.
In normal systems, it shows exactly what is stored.
However, in stealth systems, the output itself can be intercepted and modified.

2.2 Stealth Behavior

Stealth malware does not delete data.

Instead, it:

- Filters it
- Censors it
- Rewrites it
- Hides it

This makes detection much harder than simple deletion.

3 Important Terms

Malware

Malware is software intentionally designed to harm users or systems, violate privacy, or evade detection .

Rootkit

A rootkit is a type of stealth malware that hides its presence while maintaining control of the system.

Rootkits often:

- Intercept system calls
- Modify outputs
- Hide files
- Hide processes

Stealth

Stealth refers to the ability of malware to avoid detection by manipulating what the user and security tools see.

Postprocessing / Pruning

Instead of preventing an action, stealth malware often modifies the result.

Example:

- A file exists
- The system removes it from directory listings
- The user believes it does not exist

4 Why These Challenges Matter

In real systems:

- Logs can lie
- Tools can be compromised
- Views can be manipulated
- Malware hides rather than deletes

These challenges teach you:

- Not to trust a single view
- To question clean outputs
- To verify assumptions
- To think adversarially

This is the mindset of real-world security analysts.

5 Challenge 3: The Living Kernel

Narrative

Year 2147.
The bunker is no longer merely corrupted.
Power draw is steady. Cooling cycles pulse at regular intervals. Sensors indicate activity.
Yet the system monitors report:

No active tasks.

This is no longer a question of missing data.
Something is **running**.
Something is **alive**.
You are tasked with proving the anomaly exists and extracting its signature.

Mission Objective

Your goal is to locate the running anomaly and recover its unique signature.
The signature will appear in the form:

`FLAG{...}`

Rules

- This is a beginner-to-intermediate challenge.
- Copy–paste is allowed.
- You may experiment.
- You should not assume system tools are honest.

You May Use the Following Commands

- `ps`
- `top`
- `pgrep`
- `ls /proc`
- `cat /proc/<PID>/cmdline`
- `cat /proc/<PID>/environ`

New Concepts

This challenge introduces several new ideas.

Process A process is a running instance of a program. Each process has a unique identifier called a PID.

PID (Process ID) A PID is a number used by the operating system to track a running program.

`/proc` `/proc` is a virtual filesystem that exposes live system information. Each running process has a directory named after its PID.

`cmdline` This file shows how a process was started.

`environ` This file contains the environment variables of a process.

Stealth Stealth refers to the ability of software to hide its presence by modifying what tools show.

Important Note

One of the core ideas of this challenge is:

> **Do not trust a single view of reality.**
>
> Some tools may lie. Some outputs may be filtered. Some results may be pruned.
> If one view looks too clean, seek another.

Where to Begin

You are encouraged to start with:

```
ps
top
pgrep
```

If the system appears inactive, ask yourself:

> What does it mean for something to be alive if no tool reports it?

Success Condition

You have completed this challenge when you successfully recover the anomaly's signature:

`FLAG{...}`

Final Thought

This challenge is not about memorizing commands.
It is about learning how to think:

- What assumptions am I making?
- What if this tool is lying?
- What is another way to observe the system?

6 Challenge 4: Encrypted Whisper — Learning Notes

What This Challenge Is About

In the previous challenges, you encountered an anomaly that:

- Hid files
- Lied about system output
- Existed as a hidden running process

In this challenge, the anomaly has changed its strategy.
Instead of hiding its *existence*, it now hides its *meaning*.
This reflects a common real-world malware behavior:

Modern malware often hides what it contains, not just where it is.

Your task is no longer about proving that something exists. It is about understanding what that something *means*.

Key Learning Goals

By completing this challenge, you should understand:

- The difference between hiding data and hiding meaning
- Why malware uses encoding and compression
- How layered obfuscation works
- Why "noise" may actually be valuable information
- How analysts peel away multiple layers of transformation

Important Terms

Payload The payload is the actual content that malware is trying to protect or conceal. It is the *meaningful part* of the data.
In this challenge, the payload is the message you are trying to recover.

Encoding Encoding is a reversible transformation that changes how data looks, not what it means.

Encoding is often used to:

- Make data safe for transport
- Make binary data appear as text
- Evade simple text-based inspection

A common encoding format is Base64.

Compression Compression reduces the size of data by representing it more efficiently.

Compressed data often appears random or unreadable.

Compression is frequently used by malware because:

- It reduces size
- It obscures patterns
- It complicates inspection

Anti-Detection Techniques Anti-detection techniques are methods used by malware to avoid being understood, analyzed, or flagged.

Instead of deleting evidence, malware often:

- Encodes it
- Compresses it
- Encrypts it
- Wraps it in multiple layers

This forces analysts to perform step-by-step decoding.

Why This Matters in Security

In the real world, malware rarely stores information in plain text.

Instead, it uses layers of transformation:

$$\textit{Original Data} \rightarrow \text{Compressed} \rightarrow \text{Encoded} \rightarrow \text{Embedded}$$

Each layer makes casual inspection harder.

Your job as an analyst is to:

- Recognize transformations
- Identify what kind of transformation was applied
- Reverse them safely
- Extract meaning

Takeaway

This challenge teaches an important shift in mindset:

> **If something looks like meaningless noise, that may be intentional.**

Security analysis is not about guessing.
It is about:

- Recognizing patterns
- Identifying transformations
- Undoing them methodically
- Refusing to assume that unreadable means unimportant

7 Challenge 5: Survivor Protocol — Learning Notes

What This Challenge Is About

In earlier challenges you dealt with an anomaly that could:

- hide files,
- lie through tooling,
- exist as a running process,
- and conceal its messages.

In this challenge, the anomaly escalates again:

> **It resists removal.**

You are no longer just finding hidden artifacts. You are investigating *behavior over time*.

If you terminate the anomaly and it returns, that is not magic. That is a real security concept: **persistence**.

Key Learning Goals

By completing this challenge, you should understand:

- What persistence means in malware behavior
- How a watchdog/monitor process can enforce persistence
- Why relying on a single tool can be dangerous
- How to validate claims using multiple system views
- How analysts use logs and process metadata as evidence

Important Terms

Persistence Persistence is the ability of a malicious component to remain active even after attempts to remove it. It can be implemented in many ways (startup scripts, schedulers, services, watchdogs).

In this lab, persistence is demonstrated by an anomaly that returns after termination.

Monitor / Watchdog A watchdog (also called a monitor) is a process that supervises another process. If the target disappears, the watchdog restores it.

Watchdogs are common in:

- legitimate systems (high availability, self-healing),
- and malicious systems (persistence, resilience).

Compromised Tooling (Cross-Checking) In incident response, tools can lie:

- a binary may be replaced,
- output may be filtered,
- environment variables may change behavior,
- PATH order may redirect commands.

Therefore, analysts **cross-check**: if one tool reports "nothing," you confirm using another tool or another system view.

Process Metadata Processes have identifying information that can be used for investigation:

- PID (process ID): the unique number for a running process
- PPid (parent PID): who started the process
- cmdline: how the process was launched
- environ: environment variables available to that process

This metadata can reveal relationships and intent.

Why This Matters in Security

Persistence is one of the most important characteristics of real malware. Many incidents are not about *finding a file* — they are about understanding a system that:

- repairs itself,
- recreates removed components,
- restores compromised state,
- and resists cleanup.

This challenge trains you to think like an analyst:

If something returns, ask: "what mechanism is restoring it?"

Takeaway

The goal is not to brute-force commands.

The goal is to learn a pattern:

Symptom (it returns) → *Hypothesis (persistence)* → *Evidence (monitor/logs/process data)*

This pattern generalizes directly to real-world incident response.